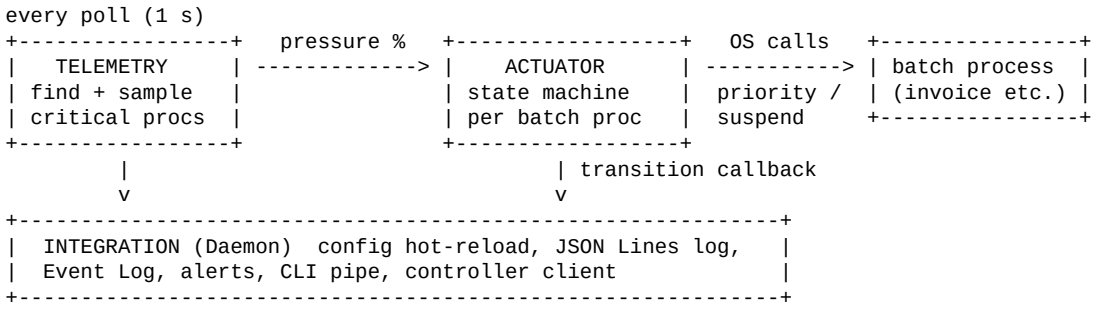
Telemetry / Monitoring Module

Module owner: **Prakul** | Logistics server resource monitor & priority throttler (Windows, C++17)

1. What this module does

Telemetry is the part of ThreadHaul-Daemon that **observes**. Once per polling interval it finds every configured critical process by executable name, measures its CPU usage from kernel+user time, its memory working set and its I/O rate, and turns that into the single **pressure %** that drives the Actuator. It survives a target that is not running yet, crashes, or restarts under a new PID. It also contains the **predictive (adaptive) thresholds**, the threshold-event tracker, the daemon's own self-monitoring, and the two dummy test applications used for the demo.

2. Where it fits in the system



Thresholds (defaults, all configurable): throttle at 85% for 3 samples, suspend at 90% for 5 samples after throttling, recover below 60% for 5 samples, 1 sample per second.

3. Files in this module

File	Purpose
Source/Telemetry/CpuMath.h/.cpp	CpuPercentFromTicks(): converts CPU-time deltas into CPU% with per-core or whole-system normalisation.
Source/Telemetry/ProcessMonitor.h/.cpp	SnapshotProcesses, FindByName, TrackedProcess (per-name sampler with lifecycle events), SystemMemoryLoadPercent, SelfMonitor.
Source/Telemetry/ThresholdTracker.h	Counts consecutive samples per threshold and reports when a streak completes (for the threshold_event log).
Source/Telemetry/AdaptiveThresholds.h/.cpp	Least-squares trend + forecast that lowers thresholds while load is rising.
Source/Simulators/critical_simulator.cpp	Dummy 'dispatch API': burns CPU following a load schedule, can simulate a crash.
Source/Simulators/batch_simulator.cpp	Dummy 'invoice job': burns CPU continuously and prints progress and its priority each second.
Source/Tests/TelemetryTests.cpp	48 unit checks: CPU math, tracker, adaptive thresholds, real process discovery and crash detection.

4. Key formulas

CPU% (per core) = 100 x [(kernel$_t$ + user$_t$) - (kernel$_{t-1}$ + user$_{t-1}$)] / (wall$_t$ - wall$_{t-1}$)
Pressure % = max(CPU%, 100 x memLoad / memLimit, 100 x workingSet / wsLimit, 100 x ioRate / ioLimit) (memory/I-O terms only when enabled)
Adaptive offset = min(slope x horizon + controllerBias, max_adjust_percent, high - low - 1), applied only while slope > 0 and forecast >= high

5. Windows APIs and libraries used

API / library	Where	What it is used for
CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS) + Process32FirstW / Process32NextW	ProcessMonitor.cpp	One snapshot per poll lists every process with its exe name and PID; FindByName matches names case-insensitively.
OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION | SYNCHRONIZE)	ProcessMonitor.cpp	Least-privilege handle that is enough to read times, memory and I/O and to wait for exit. ERROR_ACCESS_DENIED is reported once as access_denied.
GetProcessTimes	ProcessMonitor.cpp	Returns kernel and user CPU time in 100 ns units; the difference between two polls is the CPU used in that interval.
GetSystemTimeAsFileTime	ProcessMonitor.cpp	Wall-clock time in the same 100 ns units, giving the interval length.
GetProcessMemoryInfo (psapi)	ProcessMonitor.cpp	Working-set size of the critical process (memory trigger, memory_mb in logs).
GetProcessIoCounters	ProcessMonitor.cpp	Read + write + other bytes transferred; delta / seconds = I/O MB/s (I/O trigger).
WaitForSingleObject(handle, 0) + GetExitCodeProcess	ProcessMonitor.cpp	A signalled handle means the process ended; exit code 0 = exited, non-zero = crashed.
GlobalMemoryStatusEx	ProcessMonitor.cpp	System-wide memory load % for the memory-pressure trigger.
GetSystemInfo (dwNumberOfProcessors)	Daemon.cpp	Logical core count used for normalisation.
GetProcessHandleCount, GetCurrentProcess	ProcessMonitor.cpp	Self-monitoring of the daemon's own footprint.

6. Main functions written in this module

Function	What it does
CpuPercentFromTicks(dProc, dWall, cores, mode)	PerCore: 100 x dProc / dWall (one busy core = 100%, clamped to 100 x cores). TotalSystem: that value / cores, clamped to 100. Zero wall time returns 0.
SnapshotProcesses() / FindByName()	Builds the per-poll process list and returns all PIDs with a given exe name.
TrackedProcess::Poll(snapshot, samples, events)	1) drops instances whose handle is signalled and emits Exited / Crashed; 2) opens new instances and emits Acquired, or Restarted (with previousPid) if the old one just ended; 3) samples every live instance.
TrackedProcess::SampleInstance()	Reads times, memory and I/O. The first sample after (re)acquiring is marked valid=false because there is no delta yet, so a 0% reading is never fed to the Actuator.
ThresholdTracker::Add(percent, cfg)	Updates three streak counters and returns the thresholds whose streak just reached its length (high 3, critical 5, low 5). Reset() is called whenever the Actuator changes state so events line up with actions.
AdaptiveThresholds::AddSample / Slope / Forecast	Keeps the last window_samples values and fits y = a + b x by least squares; Forecast = fitted value horizon_samples ahead.
AdaptiveThresholds::Offset(base, bias)	If load is rising and the forecast reaches the throttle threshold, offset = slope x horizon, plus any controller bias, capped at max_adjust_percent and at (high - low - 1) so the hysteresis band survives.
AdaptiveThresholds::Adjust(base, bias)	Returns the effective config: high and critical lowered by the offset, low never changed.
SelfMonitor::Sample()	The daemon's own CPU%, working set, private bytes and handle count for the self_monitor log line.
critical_simulator: Worker()	Busy-loops for *load*% of every 100 ms slice; main() steps through the --schedule phases and can ExitProcess(3) for --crash-after.
batch_simulator: Worker()	Spins flat out; the reporter prints units/s and GetPriorityClass() each second and flags a gap > 2 s as 'resumed after a pause'.

7. Robustness and safety handled here

• **Target not running at startup:** TrackedProcess simply finds no PID and keeps looking every poll; no crash, and pressure is treated as 0 so batch jobs are not throttled for nothing.

• **Crash mid-monitoring:** the held handle becomes signalled; the exit code distinguishes a clean exit from a crash and is logged.

• **Restart under a new PID:** the new instance is picked up on the next poll and logged as restarted with previous_process_id.

• **Several instances of one name:** all are sampled; the highest pressure wins and its PID is reported as critical_process_id.

• **Normalisation:** *per_core* (default) keeps 85% meaning 'most of one core' on any machine; *total* means 85% of the whole server.

8. How to demonstrate it

• `build.cmd test`: TelemetryTests must print 0 failed (it launches a real cmd.exe that exits with code 3 to test crash detection).

• Console demo (RUNNING.md 4.2): the log shows a sample line per second with cpu_percent, memory_mb, io_mb_per_sec and pressure_percent.

• Crash / restart: `critical_simulator.exe --load 100 --crash-after 10`, then start it again; look for crashed and restarted lifecycle entries.

• Adaptive: RUNNING.md 4.7, look for adaptive_threshold entries with high_percent below 85.

9. Likely review questions

Q. Why not use Performance Counters / PDH?

A. GetProcessTimes on a handle we already hold is cheaper, needs no counter instance names (which change with duplicates), and gives exact kernel+user time.

Q. Why is the first sample ignored?

A. CPU% needs two readings; feeding a fake 0% after a restart could trigger a wrong recovery.

Q. Why a linear fit and not an LSTM?

A. The literature review (Gap 2) shows learned predictors need training data and a lot of compute; a monitor running on the protected server must stay at about 1% CPU. The fit is a few dozen arithmetic operations per second.

Q. Can adaptation make recovery happen too early?

A. No. Only the throttle and escalate thresholds are lowered; the recovery threshold never moves and throttle always stays above it.

See also: README.md (overview), RUNNING.md (how to run), INTERFACES.md (cross-module contract), Source/Docs/literature-review.md (design rationale).